

Comparison of Lightweight Encryption Algorithms

B.Tech Final Year Project

Rajesh Kumar Jena (B122088) Soumil Kumar (B122114) Punit Kumar (B422041)

International Institute of Information Technology
Bhubaneswar
Supervisor: Prof. Debasish Jena

May 2026

Outline

- 1 Introduction
- 2 Literature Survey
- 3 Algorithm Overview
- 4 Methodology
- 5 Results
- 6 Conclusion and Recommendations

Motivation and Problem Statement

Context

- IoT deployments require cryptography on devices with limited CPU, memory, and energy
- Conventional algorithms (AES, RSA, ECC) often exceed microcontroller resource limits
- The Mirai botnet attack highlighted risks of insecure embedded systems

Problem

- Developers sometimes deploy weakened or ad hoc security mechanisms
- Lightweight cryptography provides efficient alternatives for constrained platforms
- Need to balance security, performance, and implementation cost

This project aims to:

- Examine the evolution and design principles of lightweight cryptographic algorithms
- Evaluate selected lightweight ciphers (PRESENT, SPECK, ASCON) in terms of execution time, memory usage, and implementation cost
- Analyze their security properties based on established cryptanalytic research
- Implement and benchmark these algorithms on a Raspberry Pi Pico using Rust
- Identify design trade-offs and assess suitability for real-world constrained environments

Key Characteristics:

- **Computational efficiency:** XOR, rotations, small S-box lookups
- **Small footprint:** <2 KB Flash, <1 KB RAM
- **Energy efficiency:** avoids modular arithmetic

Design Trilemma: Security $\leftrightarrow$ Performance $\leftrightarrow$ Implementation Cost

Three Ciphers Evaluated:

- **PRESENT:** 64-bit SPN cipher, 80/128-bit keys, ISO/IEC 29192-2
- **SPECK:** Software-optimized ARX cipher, NSA-designed, highly efficient
- **ASCON:** Modern AEAD primitive, NIST SP 800-232, authenticated encryption

Design Properties Comparison

Property	PRESENT	SPECK	ASCON
Block Size	64 bits	64 bits	64 bits
Key Sizes	80, 128 bits	64–256 bits	128 bits
Structure	SPN	ARX Feistel	Sponge-Duplex
Rounds	31/32	22–34	12
Year	2007	2013	2014
Standardization	ISO/IEC 29192-2	—	NIST SP 800-232
AEAD Support	No	No	Yes

Table: Design Properties Comparison of Evaluated Ciphers

Key Properties:

- 64-bit SPN block cipher
- 80-bit and 128-bit key options
- Ultra-lightweight hardware design
- Round steps: AddRoundKey, S-box, permutation
- 31 rounds (PRESENT-80), 32 rounds (PRESENT-128)

High-Level Pseudocode

Require: 64-bit plaintext S , key K

Ensure: 64-bit ciphertext C

Generate round keys $\{K_1, \dots, K_{31}\}$

for $i = 1$ to 31 **do**

$S \leftarrow S \oplus K_i$

$S \leftarrow \text{SBoxLayer}(S)$

$S \leftarrow \text{PermLayer}(S)$

end for

$C \leftarrow S$

return C

Key Properties:

- Software-optimized ARX cipher
- Feistel-like word-based structure
- Efficient on microcontrollers
- Word sizes: 16, 24, 32, 48, 64 bits
- SPECK-64/128: 27 rounds

High-Level Pseudocode

Require: Two n -bit words (L, R) , keys $\{K_0, \dots, K_{T-1}\}$

Ensure: Ciphertext (L, R)

for $i = 0$ to $T - 1$ **do**

$L \leftarrow \text{ROR}(L, \alpha)$

$L \leftarrow (L + R) \bmod 2^n$

$L \leftarrow L \oplus K_i$

$R \leftarrow \text{ROL}(R, \beta)$

$R \leftarrow R \oplus L$

end for

return (L, R)

Key Properties:

- NIST Lightweight Cryptography Standard (2025)
- Sponge-duplex construction
- 320-bit internal state (5×64 -bit words)
- Supports authenticated encryption and hashing
- 12 rounds for permutation

High-Level AEAD Pseudocode

Require: Key K , nonce N , data A , plaintext P

Ensure: Ciphertext C , tag T

Initialize 320-bit state with K, N, IV

Absorb associated data A (apply p^a -permutation, 12 rounds)

Encrypt: $C_i \leftarrow \text{rate} \oplus P_i$

Apply p^b -permutation after each block (6 rounds)

Finalize with key injection

Derive tag T from state

return (C, T)

Performance Metrics

- **Execution Time:** ns per block (hardware timer on RP2040)
- **Cycles Per Byte (CPB):** Platform-independent efficiency metric
- **Throughput:** Mbps for comparing operational speed
- **Memory:** ROM/RAM footprint analysis

Security Metrics

- Key sizes and brute-force resistance
- Resistance to known cryptanalytic attacks (differential, linear, rotational)
- Structural robustness of each design

Hardware Environment

- Raspberry Pi Pico (ARM Cortex-M0+, 133 MHz, 264 KB SRAM, 2 MB Flash)
- Hardware timer for cycle-accurate measurements

Software Environment

- Rust `no_std` embedded implementation
- `rp2040-hal` for hardware timer access
- Release mode with LTO and size optimization
- `cargo-embed` for on-target debugging

Measurement Procedure

- 1000+ iterations per benchmark with warm-up runs
- Interrupts disabled during critical timing sections
- Cross-validation with official test vectors

Block Cipher Benchmarks (Raspberry Pi Pico)

Algorithm	ns/block	CPB	Mbps
PRESENT-80	23,665	2,958	0.34
PRESENT-128	24,542	3,068	0.33
SPECK-64/96	336	42	23.81
SPECK-64/128	338	42	23.67
ASCON-128	2,117	132	7.56

Table: Block Cipher Benchmarks on Raspberry Pi Pico

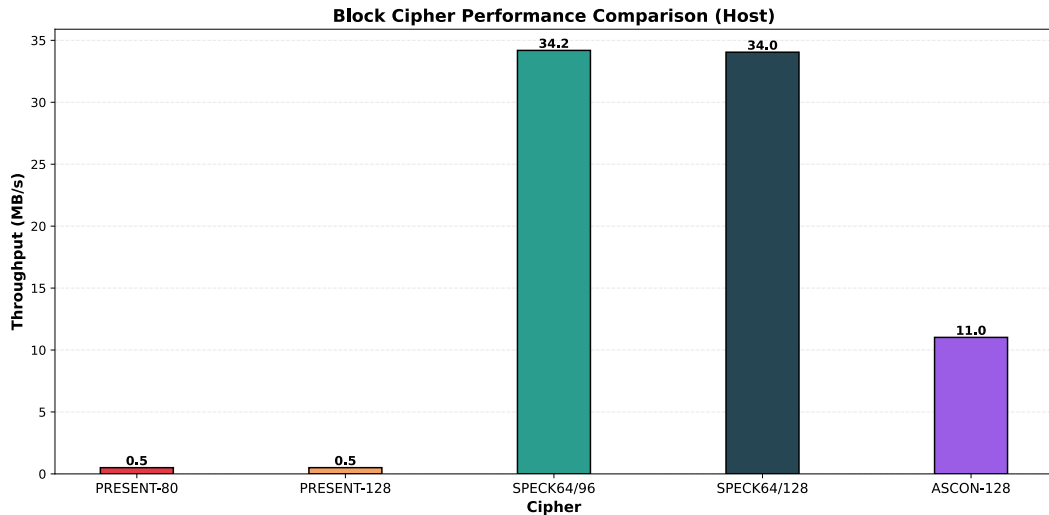
Key Finding: SPECK achieves 23.67 Mbps – approximately **70x faster** than PRESENT (0.34 Mbps) and **3x faster** than ASCON (7.56 Mbps).

Cipher	Key	State	ROM	RAM
PRESENT-80	80 bits	8 bytes	~2 KB	~16 B
PRESENT-128	128 bits	8 bytes	~2.5 KB	~16 B
SPECK-64/96	96 bits	16 bytes	~1 KB	~32 B
SPECK-64/128	128 bits	16 bytes	~1.2 KB	~32 B
ASCON-128	128 bits	40 bytes	~3 KB	~40 B

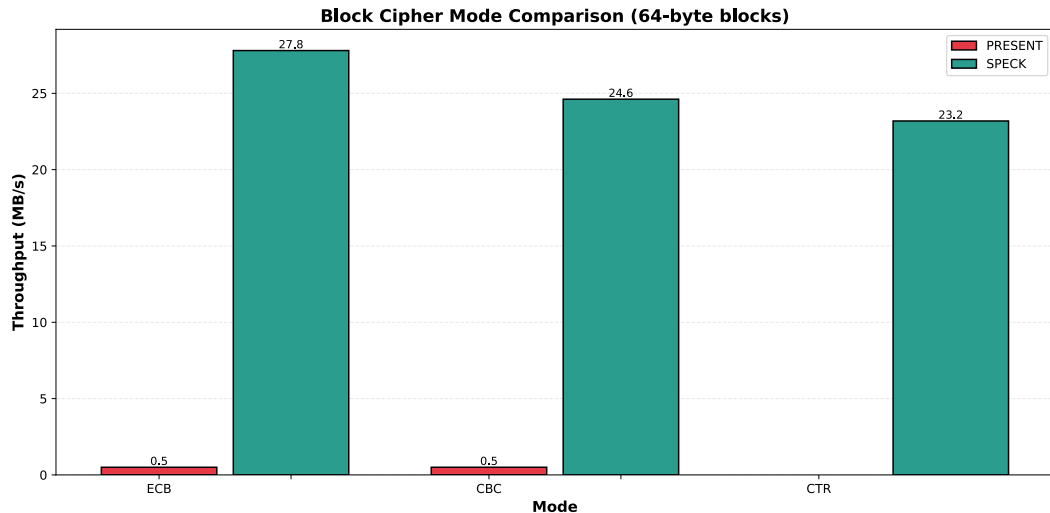
Table: Memory Footprint Comparison

Key Finding: SPECK has the smallest ROM footprint (~1 KB) due to ARX-only operations (no lookup tables).

Block Cipher Performance Comparison



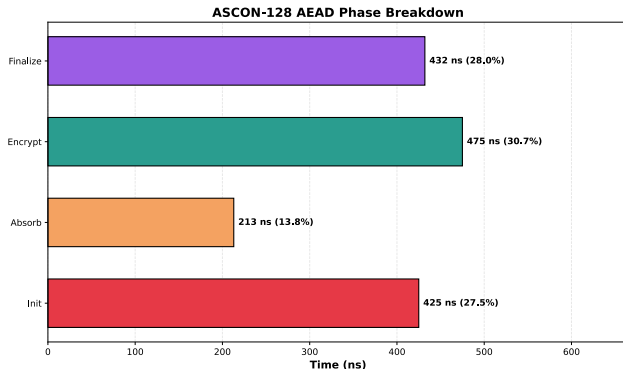
Mode Comparison: ECB vs CBC vs CTR



ASCON Phase Breakdown

Phase	Cycles
Initialization	427
Absorb (AD)	216
Encrypt	464
Finalize	422
Total	1,529

Table: ASCON AEAD Phase Breakdown



Key Findings:

- **SPECK**: Best raw throughput – optimal for bulk encryption
- **ASCON**: Built-in AEAD – recommended for security-critical IoT
- **PRESENT**: Lowest hardware gate count – suited for RFID-class devices

Trade-off Observation: ASCON's initialization overhead (427 cycles) becomes amortized at larger payload sizes, reducing the performance gap between SPECK and ASCON.

ASCON

- NIST LWC Standard (SP 800-232)
- 128-bit security level
- Built-in AEAD support

PRESENT

- ISO/IEC 29192-2 standardized
- 80/128-bit key options
- Ultra-low gate count

SPECK

- No official standard
- NSA origin (debated)
- Confidentiality only

Scorecard Comparison

Metric	PRESENT	SPECK	ASCON
Throughput (Mbps)	0.34	23.67	7.56
CPB (cycles)	2,958	42	132
ROM (KB)	2.5	1.2	3.0
RAM (bytes)	16	32	40
Authenticated Encryption	No	No	Yes
NIST Standard	No	No	Yes (SP 800-232)
ISO Standard	Yes (29192-2)	No	No
Hardware Efficiency	Excellent	Good	Good
Software Efficiency	Poor	Excellent	Good

Table: Scorecard Comparison: PRESENT, SPECK, ASCON

All 6 Practical Recommendations:

- ❶ **Real-time sensor data encryption:** SPECK-64/128 in CTR mode
 - 23.67 Mbps throughput, 32 bytes RAM, ideal for high-bandwidth sensor streams
- ❷ **Secure IoT communication with authentication:** ASCON
 - 7.56 Mbps sufficient for most IoT, built-in AEAD, NIST standardized
- ❸ **Hardware-constrained RFID tags:** PRESENT
 - Ultra-low gate count, 0.34 Mbps adequate for low-bandwidth tag communications
- ❹ **Firmware authentication:** ASCON-hash
 - 320-bit state and 12-round permutation for anti-tampering verification
- ❺ **Mixed microcontroller deployments:** SPECK + ASCON hybrid
 - SPECK for bulk data encryption, ASCON for authentication tags
- ❻ **Legacy system integration:** PRESENT-128 in CBC mode
 - Backward compatibility with 128-bit security

Thank You

Rajesh Kumar Jena (B122088)

Soumil Kumar (B122114)

Punit Kumar (B422041)